

+ 26

My DAG Ate All My Cores

Identifying and Resolving Build Graph Bottlenecks

FLORENT CASTELLI



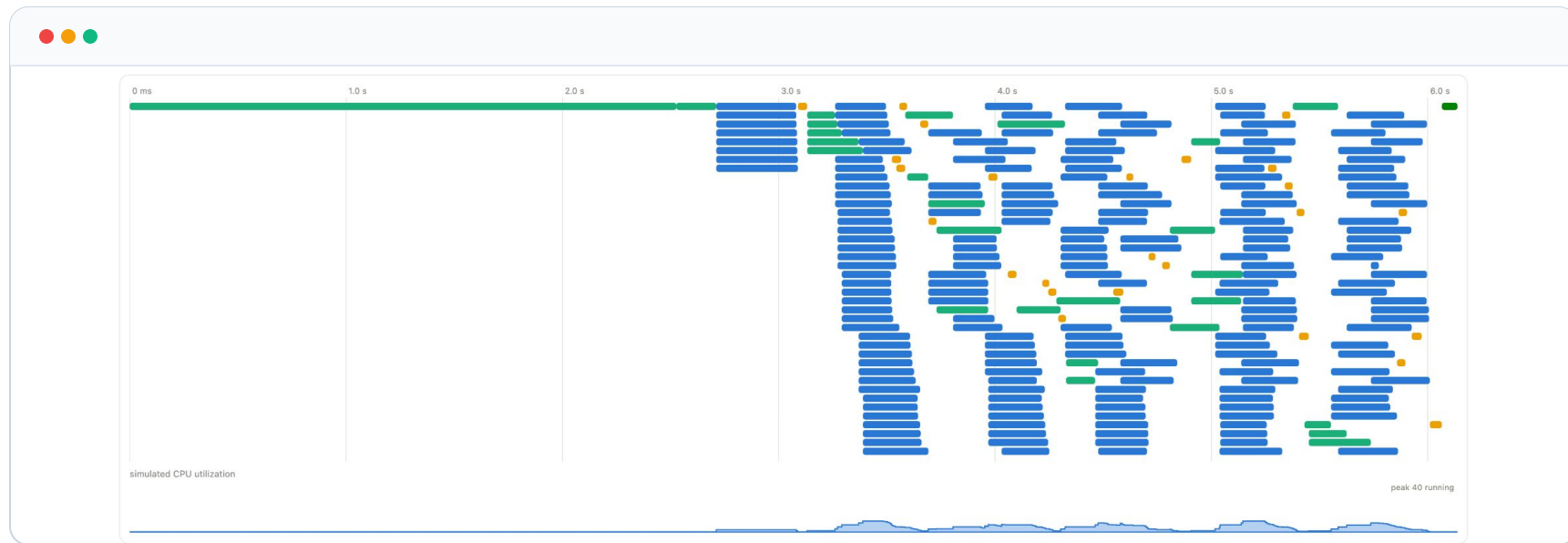
20
26



You bought **128 cores**. Your build uses **40** at most.

The machine is not the problem.

The graph is.



What you'll leave with

01 Build Graph Vision

A way to see your build as a graph, not a folder of files

02 Hardware Metrics

Three numbers that tell you whether hardware will help

03 Graph Optimization

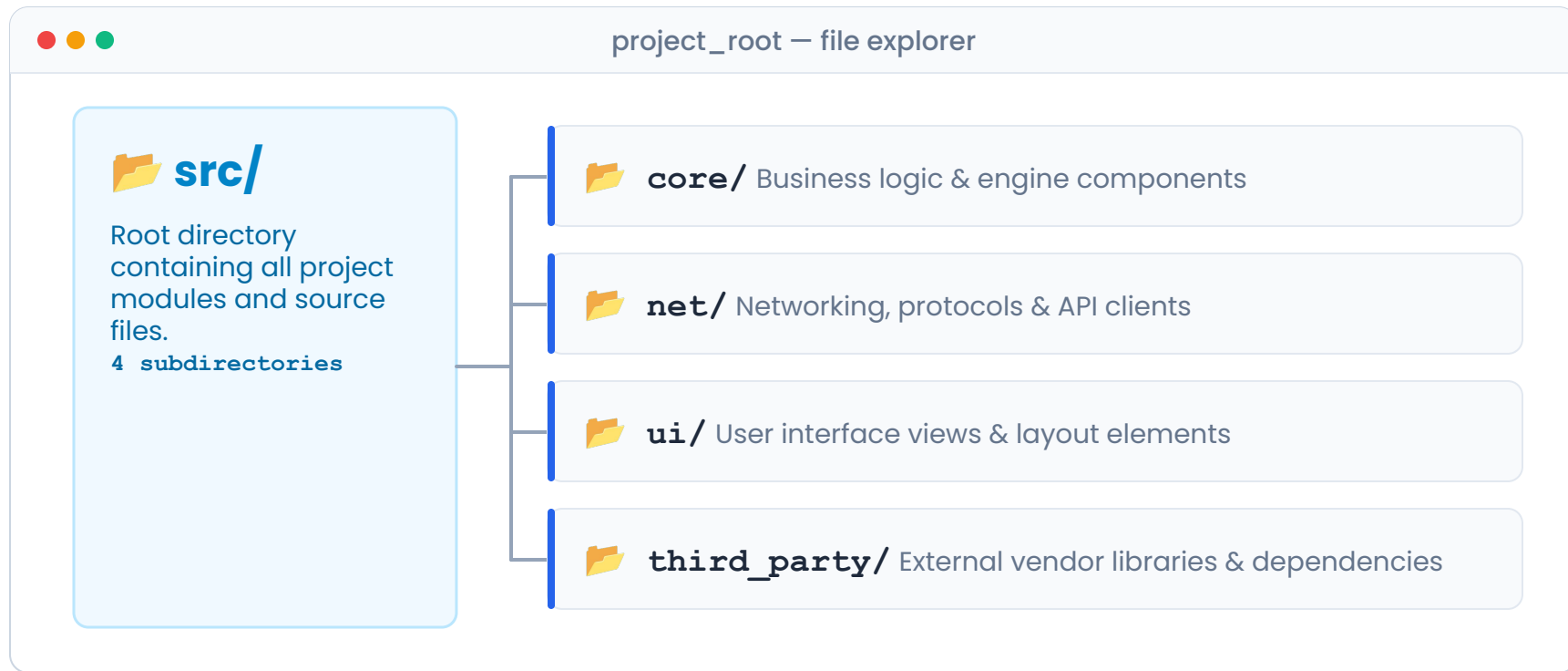
Concrete edits that change the graph's shape

04 Regression Prevention

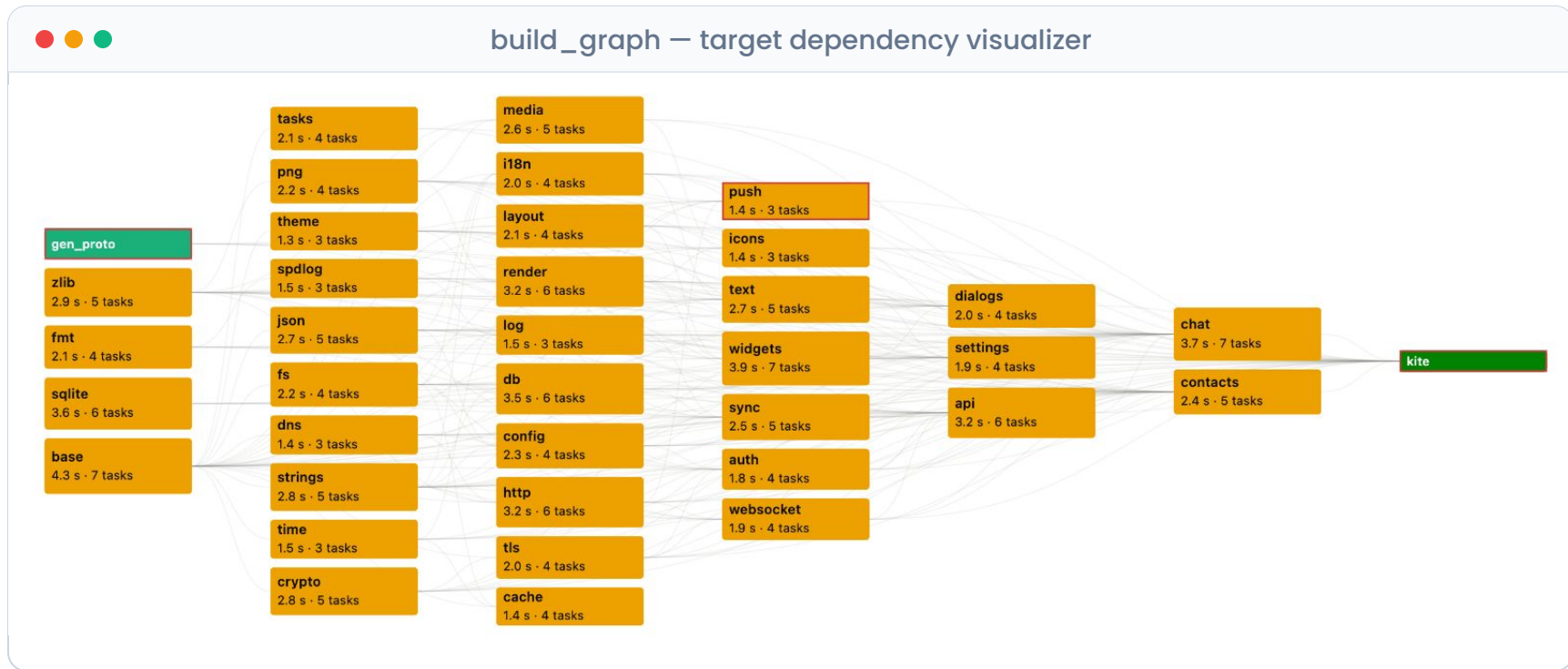
A CI check so it never regresses

The graph you didn't know you wrote

How you see your codebase (File Hierarchy)



How your build sees it



A build system computes **exactly one thing**

Nodes

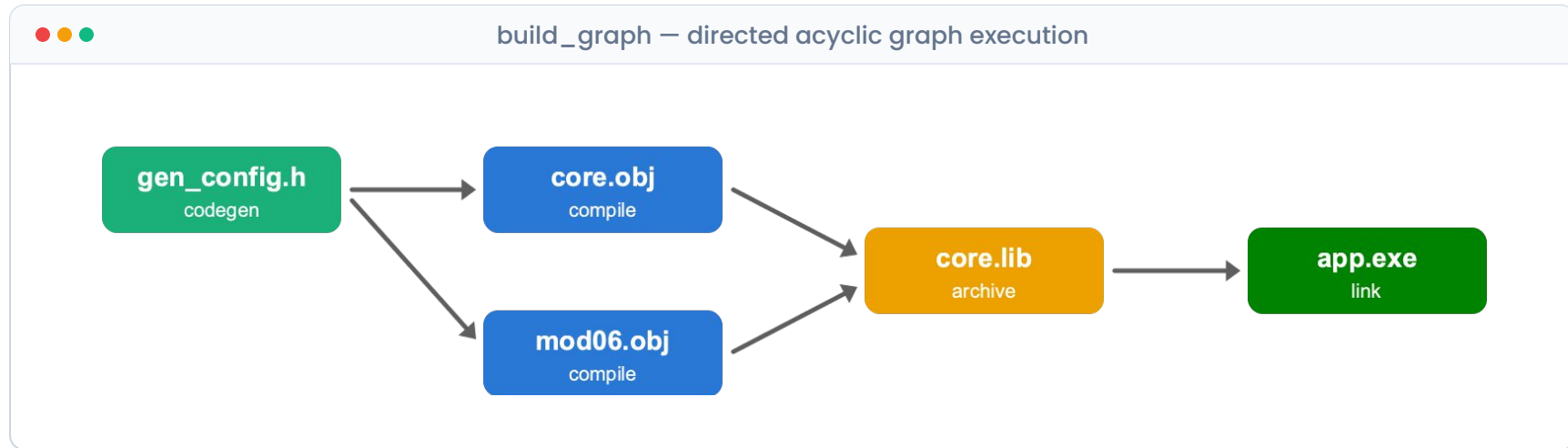
Things to produce:
Objects, archives, generated
headers, & binaries

Edges

Dependency rules:
"This must exist before that can
start"

Execution vs Setup

Rules, variables & toolchains
determine **how** the graph is built,
not **what** gets executed.



Familiar pain, restated as graph problems

"Why did touching one header rebuild 400 files?"

→ **High fan-in node**

"Why is my 96-core CI box idle half the time?"

→ **Long critical path**

"Why is the incremental build slower than it should be?"

→ **Edges that shouldn't exist**

"Why does link order matter?"

→ **You have a cycle you're pretending isn't there**

What you write

CMakeLists.txt

```
add_custom_command(  
    OUTPUT generated/mod06/mod06_gen.h  
    DEPENDS libs/mod06/mod06_gen.h.in  
            generated/core/core_gen.h)  
  
add_custom_target(gen_mod06 DEPENDS  
generated/mod06/mod06_gen.h)  
  
add_library(mod06 STATIC  
    libs/mod06/mod06_fn0.cpp  
    ...  
    libs/mod06/mod06_fn7.cpp)  
  
add_dependencies(mod06 gen_mod06)  
target_link_libraries(mod06 PUBLIC core)
```

Code Generation Rule

`add_custom_command` creates headers dynamically before compilation begins.

Target Node

`add_library` groups source files into a discrete build artifact (`mod06`).

Explicit Graph Edges

`add_dependencies` & `target_link_libraries` define strict execution ordering in the DAG.

What runs (Ninja Build File)

```
build.ninja

build generated/mod06/mod06_gen.h: CUSTOM_COMMAND
mod06_gen.h.in generated/core/core_gen.h

build CMakeFiles/mod06.dir/.../mod06_fn2.cpp.obj:
CXX_COMPILER...
mod06_fn2.cpp ||
cmake_object_order_depends_target_mod06
( * 8, one per source file )

build mod06.lib: CXX_STATIC_LIBRARY_LINKER...
mod06_fn0.cpp.obj ... mod06_fn7.cpp.obj || core.lib
gen_mod06
```

3 declarations → 10 edges

Custom Command Rule

Generates dynamic header files before compilation starts, ensuring clean header dependency ordering.

Parallel Compilation

Expands into 8 separate object compilation tasks running independently across available CPU cores.

3 Declarations → 10 Edges

A concise high-level build declaration expands into a precise, low-level DAG with explicit ordering constraints.

“Faster” is three different questions

01. CLEAN BUILD

CI from Scratch

Bounded by the critical path
and how wide the graph gets.

02. INCREMENTAL

You Touched One File

Bounded by the blast radius
of what you touched.

03. AFTER GIT PULL

The One You Do All Day

Bounded by the blast radius
of what everyone else
touched.

Same graph. Different metric. Different fix.

Where graphs go wrong

Five ways a graph goes wrong

01. CYCLES

The graph that isn't a DAG at all

AFFECTS: EVERYTHING



not a DAG

nothing else applies

02. FAT NODES

One task that dominates everything

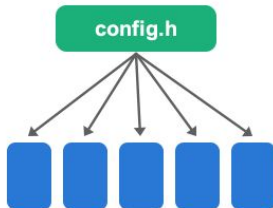
CLEAN BUILD



03. HUB NODES

One header that everything depends on

INCREMENTAL



04. DEEP PATHS

A chain no hardware can shorten

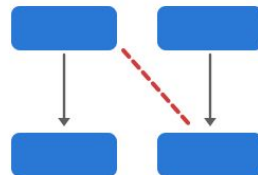
CLEAN BUILD



05. ACCIDENTAL EDGES

Dependencies nobody meant to declare

AFFECTS: BOTH



First, make it a DAG

01. CYCLES ARE UNACCEPTABLE

Ninja fails immediately on cyclic dependencies

```
ninja: error: dependency cycle:  
a.txt -> b.txt -> c.txt -> a.txt
```

CMake permits cycles only between **STATIC** libraries by linking components twice: **A B A B**

02. REFUSE & BREAK CYCLES

1. Explicitly refuse cycles in CMake:

```
set_property(GLOBAL PROPERTY  
GLOBAL_DEPENDS_NO_CYCLES ON)
```

2. Refactor the graph:

- **Merge targets** — they were one component anyway
- **Hoist shared code** — extract into a third shared target

What you declared



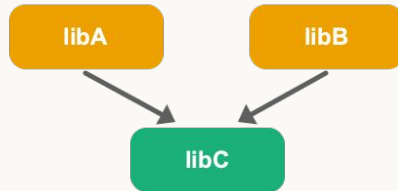
CMake links it twice:
A B A B

Merge — they were one component



Broad mutual surface —
the split was fiction

Hoist what they share



A few shared types — and C was
the concept you were missing

No topological order → no critical path, no layers, no levels.

Fat nodes

01. DEFINITION

Includes the World

One translation unit that includes the world.

02. IMPACT

Costs you twice:

- It is slow every time it compiles
- It is very likely to be on the critical path

Symptom: one bar in the timeline that is longer than the tail of the build

Hub headers

01. UBIQUITOUS HEADERS

Every Project Has One

`config.h`, `common.h`, `types.h`

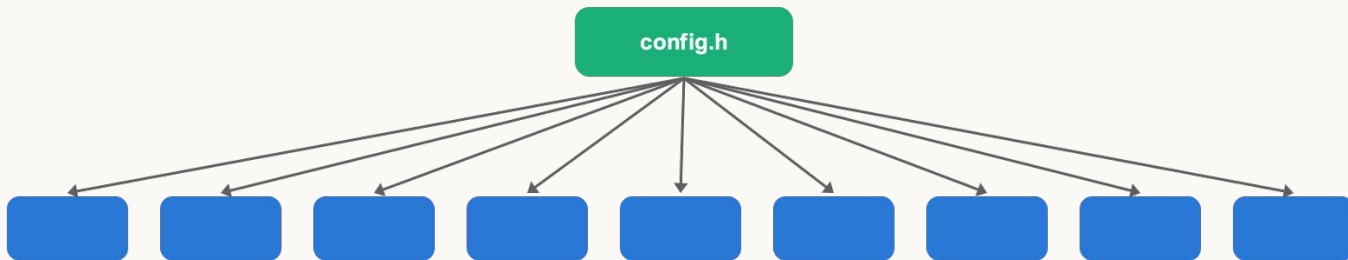
Central headers included across almost every translation unit in the codebase.

02. TRANSITIVE REBUILD SET

The Hidden Cost

The cost is not the header itself, but its blast radius:

- Recompile every TU that reads it
- Relink every archive behind those TUs



Symptom: a one-line edit costs a coffee break

Putting a price on a header

01. INVERTED DEPENDENCIES

How NinjaScope Measures Cost

NinjaScope inverts the discovered build graph to attribute wall time directly to headers:

header

↓ **tasks that read it**

↓ **transitive rebuild set**

↓ **CPU work of that set**

= wall time at your actual -j

02. SAMPLE MEASUREMENTS

Measured Header Impact

Full build time: **19.8 s**

include/work.h

200 readers · 226 tasks **16.1 s**

libs/core/core.h

9 readers · 35 tasks **3.4 s**

libs/mod06/mod06.h

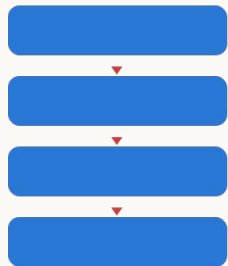
9 readers · 25 tasks **2.5 s**

Deep paths: the floor you cannot break

01. ABSOLUTE LIMIT

Longest Chain Constraint

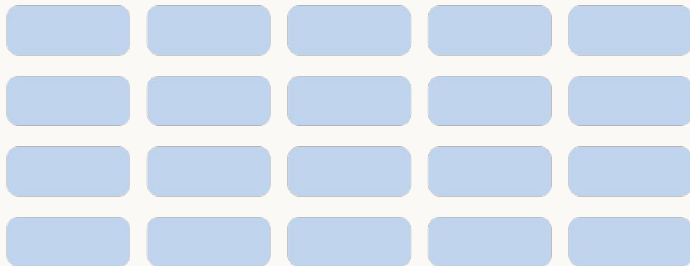
Wall time can never go below the longest chain in the build graph.



02. INEFFECTIVE FIXES

What Cannot Break It

- Not with more cores
- Not with a faster compiler (only shortens link)
- Not with distributed builds



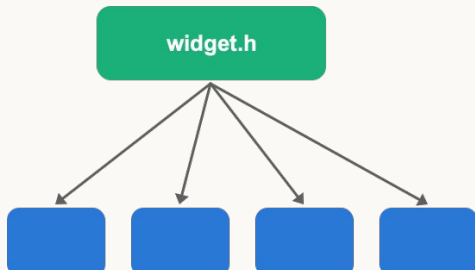
The critical path is a hard floor.

Accidental edges

01. UNINTENDED DEPENDENCIES

Declared Without Meaning To

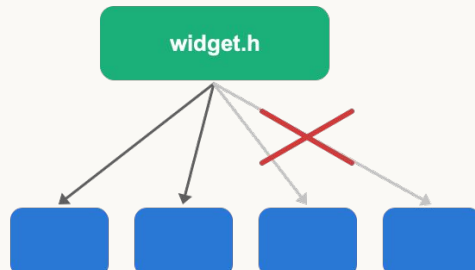
Every **#include** is an edge you declared in your build graph, usually created without realizing its full impact.



02. COMMON SOURCES

How Edges Accumulate

- Include for a type you only hold by pointer
- Include kept after code using it was deleted
- A convenience umbrella header



Easy to write. Paid for on every single build, forever.

The pathology that hides from you

01. IDENTICAL INPUTS

Same Work, Same Tasks

Same sources. Same 253 tasks. Same durations.

02. HIDDEN DIVERGENCE

The Critical Path Trap

Wall time appears nearly identical today, but critical path length differs by over 2x.

PRECISE GRAPH

Wall time: 19.8 s | Critical path: **6.9 s**

6.9 s

length

10

tasks

76%

codegen

19%

compile

3%

archive

2%

link

COARSE GRAPH

Wall time: 20.4 s | Critical path: **14.2 s**

14.2 s

length

20

tasks

57%

compile

35%

codegen

7%

archive

1%

link

On today's machine: a 3% difference (invisible). On tomorrow's: 2x faster, forever.

Measure it

Stop guessing. Three numbers.

01. TOTAL WORK

Total Work

Sum of every task's duration

02. LONGEST CHAIN

Critical Path

The longest dependent chain

03. RATIO

Avg Parallelism

$\text{total work} \div \text{critical path}$

If average parallelism < your core count, hardware will not help you.

19.8 s

WALL TIME
as built

4 m 22 s

TOTAL WORK
2 m 38 s of it uncontended

13×

AVG PARALLELISM
peak 16 tasks at once - ~1.5 GB
resident

6.9 s

CRITICAL PATH
35% of wall time

38×

MAX SPEEDUP
 $\text{total work} \div \text{critical path}$

253

TASKS
10 on the critical path

Computing the critical path

01. FORWARD PASS

Earliest Finish Time

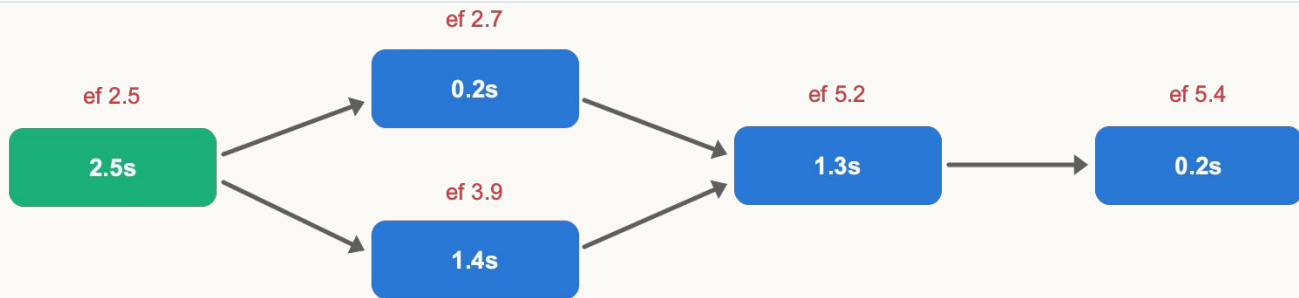
Topological sort, then one forward pass:

```
earliest_finish[n] = max(earliest_finish[deps]) +  
duration[n]
```

02. BACKWARD PASS

Slack & Task Flexibility

A backward pass gives slack: how late each task could start without moving the finish line.



Linear in nodes + edges. A 100k-node graph takes milliseconds.

The tools you already have

01. GRAPH VISUALIZATION

ninja -t graph

The declared DAG, exported as GraphViz

02. DEPENDENCY PARSING

ninja -t deps

What each object actually read during the build

03. COMMAND INSPECTION

ninja -t commands

The exact command line executed for any output

04. BUILD LOGGING

.ninja_log

Start/end timestamps, every edge, every build

Built directly into Ninja. Zero install, immediate diagnostic data.

Reading .ninja_log

01. LOG STRUCTURE & FIELDS

`start_ms` `end_ms` `mtime` `output` `command_hash`

02. BUILD HISTORY

Append-Only Record

- Append-only across builds — it is a history, not a snapshot
- Recompaction destroys run structure

03. OMITTED METRICS

Unrecorded Context

- Records neither `-j` nor the machine it ran on
- Cannot infer hardware thread count or core contention from logs alone

A clean full build is the gold standard measurement.

Seeing it: the timeline

01. VISUAL STRUCTURE

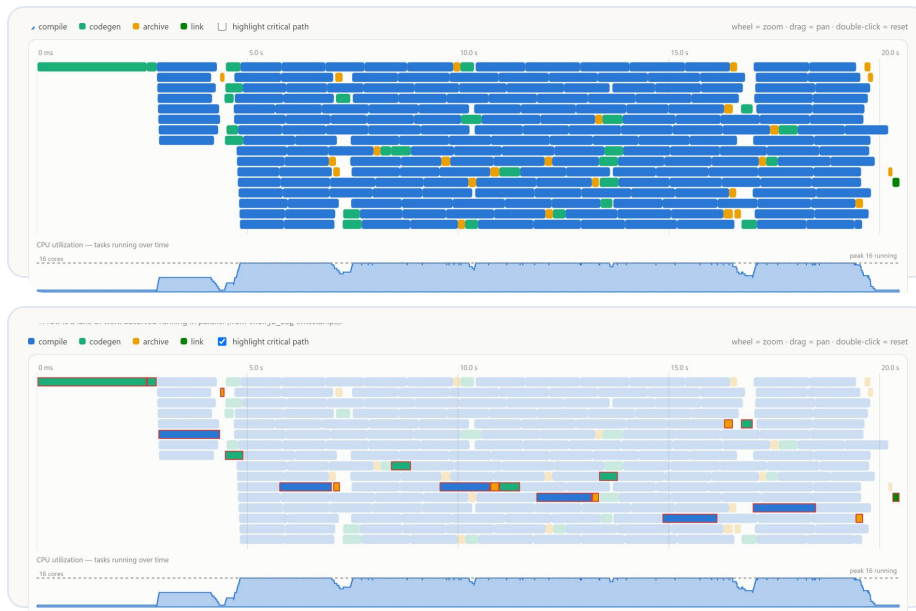
Lanes & Tasks

One row per parallel lane. One bar per task, colored by kind.

02. ANALYSIS OVERLAYS

Critical Path & Cores

- Dim everything off the critical path
- Utilization strip: active core count at each instant



The gaps are the story.

Seeing where the minutes go

01. VISUAL SCALE

Box Area = Build Time

Visual weight directly reflects total build duration for instant hot-spot detection.

02. HIERARCHICAL DRILL-DOWN

Directories & Tasks

- Directories contain target groups
- Targets break down into individual tasks



Answers "what is this build even spending time on" in one glance.

The what-if: how many cores could this build use?

01. CORE SIMULATION

Replay the Real DAG

Replay the measured durations from your build at any target core count.

02. SANITY CHECK INVARIANTS

3 Live Verification Rules

1 core → Total work

∞ cores → The critical path

your -j → Within ~1% of actual wall time

Three sanity properties you can check live.



The speedup curve, and its knee

01. PREDICTED SCALING

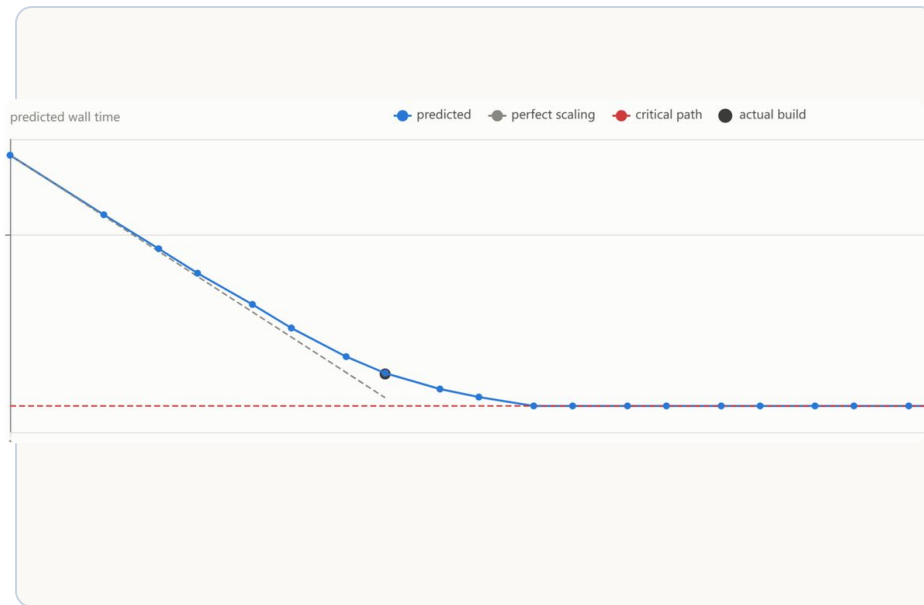
Wall Time vs. Cores

Predicted wall time mapped against core count on a log-log scale.

02. BENCHMARK BASINES

Scaling vs. Limits

- **perfect scaling** ($\text{work} \div \text{cores}$)
- **critical path** floor limit



The knee is where buying cores stops paying.

The measurement lies to you (a little)

01. RESOURCE CONTENTION

Measured in a Crowd

Every duration in `.ninja_log` reflects 16 compiles competing for memory bandwidth, cache, disk, and turbo headroom.

02. LOG LIMITATION

Unrecoverable Alone-Time

- 1.0s alone vs 1.7s in crowd
- You cannot recover true standalone duration directly from log files

The same compile, measured twice

alone

1.0 s

in a crowd of

1.7 s

A build is either saturated or idle — there is nothing in between to sample.

So measure it

01. EMPIRICAL BENCHMARKING

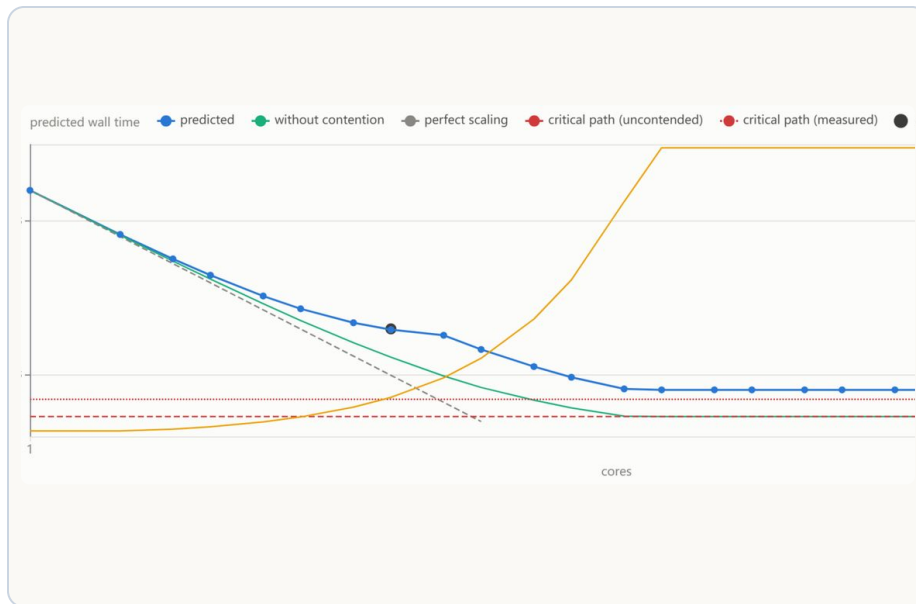
Re-run Real Build Commands

- Alone, & at 25 / 50 / 100 / 150% job load
- Randomised order to catch thermal drift
- Real background load (no synthetic spinners)

02. MEASURED DELIVERABLES

Contention Curve & Tax

- **Contention curve:** Real scalability mapping
- **Contention tax:** Extra work spent purely due to machine overcrowding



Isolate real execution to quantify the hidden tax of a crowded machine.

Change the shape

Now change the shape

01. SHORTEN PATH

Shorten the path

Break the chain of sequential work.

02. REMOVE EDGES

Remove edges

Stop declaring what you don't use.

03. COLLAPSE EDGES

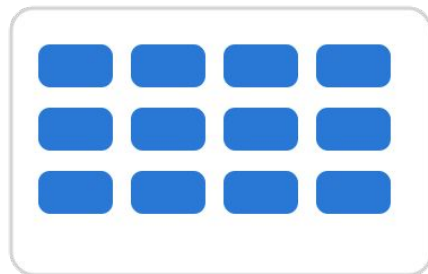
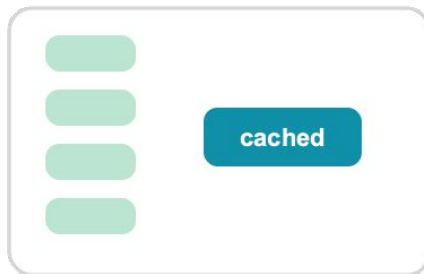
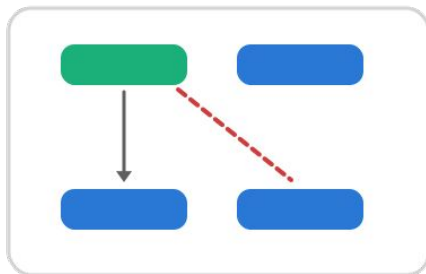
Collapse edges

Pay once, cache it for future steps.

04. WIDEN EXECUTION

Widen execution

More workers, not more cores.



Cheapest and most durable first.

Optimise on the path, not off it

01. SLACK MECHANICS

Understanding Slack

Slack = how much later a task could finish without moving the finish line.

`slack = 0` (Critical Path)

Every second saved is a second off the build.

`slack > 0` (Free Real Estate)

Speed it up and nothing happens.

02. SAMPLE ANALYSIS

253 Tasks, 262s of Work

2.6 % (10 tasks / 6.9 s)

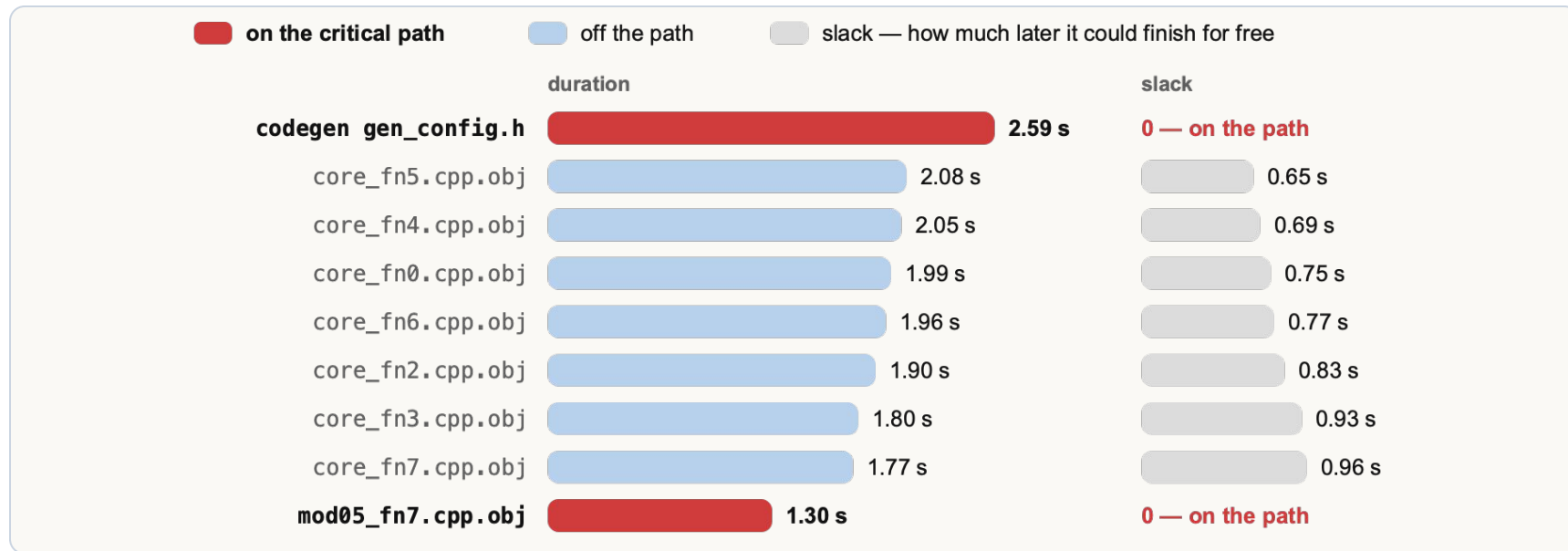
Decides the floor limit of the build.

97.4 % (243 tasks / 255.1 s)

Has somewhere to hide off the path.

Focus optimization on the 2.6% that decides the floor — the other 97.4% has somewhere to hide.

The seven slowest compiles in this build do not matter



The slowest seven compiles are all off the path. The 1.30 s one is on it — and matters more.

Splitting a hub header

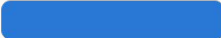
Before — one header, three unrelated concerns

String helpers + clock + log macro, included by every translation unit

	Readers	Rebuild set	Wall time until the tests can run again	
base/util.h	111	143		4.3 s 87% of a full build

After — split by concern, each TU includes only what it uses

Same code, same compiler, same machine — only the #include lines changed

	Readers	Rebuild set	Wall time until the tests can run again	
base/util_time.h	40	72		1.9 s 39% of a full build
base/util_log.h	37	69		1.7 s 35% of a full build
base/util_strings.h	34	66		1.5 s 31% of a full build

Splitting one heavy hub header into focused modules eliminates unnecessary rebuilds.

Removing edges: forward declarations and pimpl

01. FORWARD DECLARATION

```
class Widget;
```

costs nothing

Hold by pointer or reference → you only need a **declaration**, not a definition.

```
#include "widget.h"
```

Pulls its entire transitive closure into every consumer.

02. PIMPL IDIOM

Pointer to Implementation

boundary isolation

The definition stops cleanly at the `.cpp` boundary.

Edge disappears from every consumer at once.

Internal changes no longer trigger rebuild cascades across the DAG.

Forward declarations cost nothing; pimpl stops definitions at the .cpp boundary so edges disappear for all consumers.

Hide the implementation, not just the definition

01. WHAT CONSUMERS SEE

engine.h

clean interface

```
class IEngine {  
public:  
    virtual ~IEngine();  
    virtual void run() = 0;  
};  
std::unique_ptr<IEngine> make_engine();
```

A library using heavy internal dependencies should never declare them in its header.

02. WHERE THE WEIGHT LIVES

engine.cpp

isolated subtree

```
#include <heavy/everything.hpp>
```

plmpl removes one edge. An interface removes a subtree.

Heavy headers stay contained inside implementation files without polluting consumers.

Consumers get one edge to engine.h — not heavy/everything.hpp and its entire transitive closure.

PUBLIC is a promise you keep paying for

01. PRIVATE

Internal Only

stops at boundary

```
target_link_libraries(  
    engine PRIVATE heavy)
```

I use heavy. You don't need to know.

Consumers do not get its headers.

02. PUBLIC

Transitive Leak

reaches every consumer

```
target_link_libraries(  
    engine PUBLIC heavy)
```

heavy is in my public header,
so you get it too —

and so does every consumer of
every consumer.

03. INTERFACE

Forwarded Only

consumers use it

```
target_link_libraries(  
    engine INTERFACE heavy)
```

I don't use it.

Using my header makes you
use it.

The rule: PUBLIC only if the dependency appears in your public header. Otherwise PRIVATE.

Removing edges: let a tool find them

01. BATCH TOOL

include-what-you-use

batch, whole repo

Every include you don't need is an edge you're paying for.

Thorough, noisy, and will suggest things that break your build.

02. EDITOR INTEGRATION

clangd include-cleaner

incremental, in your editor

Unused includes greyed out as you type, one file at a time.

```
.clangd:  
Diagnostics:  
UnusedIncludes: Strict  
MissingIncludes: Strict
```

Tools win adoption when they enable gradual migration without requiring a global cutover.

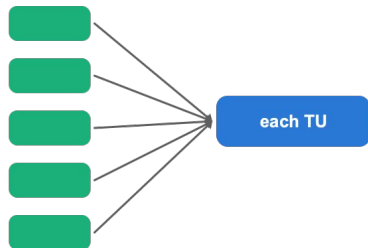
Collapsing edges: precompiled headers, and modules

01. SAME TRICK, TWO ERAS

PCH & C++ Modules

edge consolidation

- **PCH**: parse header pile once, reuse snapshot
- **C++ Module (BMI)**: compile interface once, import artifact

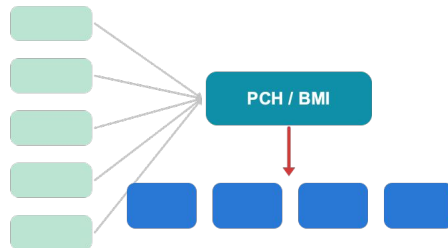


02. SHARED FAILURE MODE

Critical Path Bottleneck

serialization point

- That node becomes something every consumer waits for.
- Sample cost: **2.59 s of a 6.9 s** critical path in global codegen.



Both turn many expensive edges into one expensive node, but create a single node every consumer waits for, so measure it as a shape change.

Modules change the shape — and then pay for it

Same sources. Same libraries. Four builds of the hub:

METRIC	No modules	Scan only	Hub as module	+ import std;
Tasks	253	480	484	490
Total work	121 s	192 s	45 s	43 s
Wall time	9.7 s	13.7 s	5.2 s	5.1 s
Touch the hub	6.6 s	10.0 s	2.5 s	2.0 s
Touch one .cpp	0.5 s	0.8 s	0.35 s	0.35 s

Scanning is the price of admission. Importing is where it pays back.

Widening: more workers, not more cores

01. INFRASTRUCTURE

Distributed Builders

RBE, distcc, icecream

Offloads compilation actions from local developer hardware to remote execution nodes.

Enables massive horizontal scaling across dedicated build farm capacity.

02. EXECUTION PROFILE

Parallel Scaling Effects

zero local contention

- The critical path does not move
- The wide phase gets much wider
- **Contention on local memory disappears** — executed on separate machines.

This is the "without contention" curve. It is the remote-execution reading.

Widening has a ceiling too

01. DIMINISHING RETURNS

Scaling Limits

flattening curve

- **Slider to 8192 cores** — the curve flattens anyway.
- **Beyond saturation**, extra machines sit idle during sequential phases.
- Spare capacity remains available for other builds, requiring adequate pool sizing.

02. SYSTEM BOUNDARY

Graph Bottlenecks

structural limitation

- Hardware scaling cannot bypass DAG critical path dependencies.
- Adding workers only helps wide execution, not sequential chains.

Everything above the floor is a graph problem, and only a graph fix moves it.

The cheapest node is the one you don't build

01. CACHE SCOPE

Local & Remote Caching

work avoidance layers

`ccache` / `sccache`

Local, keyed on preprocessed input

`RBE` / `remote cache`

Shared across the whole team and CI

02. CACHE BEHAVIOR

Graph Traversal Overhead

node remains in graph

A cache hit removes node work, NOT the node itself:

- The dependency graph is still walked
- Critical path becomes cache lookups instead of compiles

Which makes hit rate the metric — and a hub header a cache-miss amplifier.

Invalidation: what counts as changed

01. TIMESTAMPS

Coarse File Tracking

touch a file → full rebuild downstream

- Triggers downstream rebuilds based on modification time.
- Touching any source file causes everything downstream to rebuild.

Executes redundant work even if contents are unchanged.

02. CONTENT HASHES

Exact Content Fingerprinting

reformat a comment → no downstream rebuilds

- Keyed strictly on actual byte contents and content hashes.
- Unchanged content → nothing downstream runs.

Drastically reduces executed work across the build graph.

The graph decides what ***can*** rebuild. Invalidation decides what ***does***.

The graph you declared vs. the graph you got

01. UNDER-DECLARED EDGES

Flaky & Unreliable

"works after a clean build"

- Missing dependency links cause non-deterministic outputs
- Leads to flaky incremental builds across workers
- Fails unexpectedly, forcing developers to clean caches



02. OVER-DECLARED EDGES

Correct, Slow, & Invisible

silent build bloat

- Unnecessary dependencies artificially extend critical paths
- Builds remain correct, but execute much slower than needed
- **Completely invisible penalty** — no test ever fails



The second kind never fails a test. That is why it survives for years.

Case study: the same build, two graphs

01. PRECISE GRAPH

38.0x Max Speedup

6.9s critical path

Tasks:	253
Total Work:	262.1 s
Wall Time:	19.8 s
Critical Path:	6.9 s
Max Speedup:	38.0x

02. COARSE GRAPH

17.6x Max Speedup

14.2s critical path

Tasks:	253
Total Work:	249.4 s
Wall Time:	20.4 s
Critical Path:	14.2 s
Max Speedup:	17.6x

Coarse graphs double the critical path (14.2s vs 6.9s) and cut max speedup in half (17.6x vs 38.0x).

What actually differed: one CMake habit

01. CMAKE LIMITATION

Directory Scope Fallback

file-level DEPENDS restriction

- Codegen declared in each library's own subdirectory.
- File-level DEPENDS on custom outputs cannot cross CMake directories.

02. IMPLICIT SERIALIZATION

Whole-Target Fallback

codegen waits for full archives

- Ordering silently falls back to target dependencies.
- **Each step waits for dependencies' archives.**
- Every layer serializes behind the previous one.

what you meant



codegen waits for the header it reads

what CMake wrote



codegen waits for the dependency's archive — every layer serialises

Nobody wrote a bad dependency. The build system filled in a conservative one.

The two critical paths, side by side

01. PRECISE PATH

Header-to-Header Flow

10 tasks, 6.9 s

- Codegen chain flows directly header to header:

`gen_config.h` → `core_gen.h` → `mod06_gen.h`
→ `mod17_gen.h` → ...

- Single compilation and archive step at end:

→ one compile → one archive → `app.exe`

02. COARSE PATH

Intermediate Detours

20 tasks, 14.2 s

- Every layer detours through compile and archive:

`gen_config.h` → `core_gen.h` → `core.obj` → `core.lib`

→ `mod06_gen.h` → `mod06.obj` → `mod06.lib`

→ `mod17_gen.h` → ...

Coarse graphs double execution tasks and time (20 tasks / 14.2s vs 10 tasks / 6.9s) by serializing behind intermediate archives.

This is not a Ninja problem

01. RECURSIVE MAKE

Directory

sync point at every dir

- Granularity picks the directory level.
- Creates a synchronization point at every subdirectory.

02. MSBUILD

Project

/m parallelises across

- Granularity picks the project level.
- /m parallelises across projects, not within one unless asked.

03. NINJA

File

fine-grained graph

- Granularity picks individual files.
- Maximizes graph precision and parallel execution potential.

Coarser granularity is a smaller graph and a longer critical path.

Shape the targets for the tool you actually use

01. MSBUILD

Project-Level Unit

prefer fewer, larger projects

- The project is the unit of scheduling.
- Many small ones multiply sync points and per-project overhead.

02. NINJA

File-Level Unit

fine-grained file targets

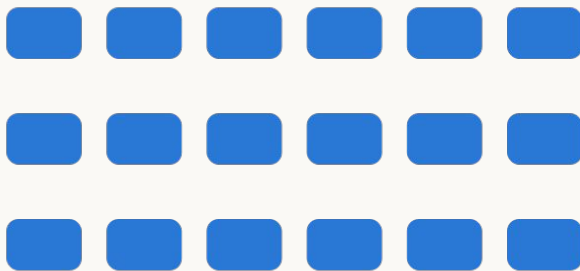
- The file is the unit; small targets cost almost nothing.
- A library per header/.cc pair is a reasonable thing to do.

project 1



sync point

project 2



The same “good architecture” advice gives opposite answers depending on your tool.

Fine grain makes dependencies checkable

01. COARSE TARGETS

Undecidable Dependencies

no crisp answer

- Question: “Does this project really need that one?”
- 400 files in a target means someone probably uses it.
- Cleanup is risky and impossible to verify automatically.

02. FINE TARGETS

Decidable & Tool-Checked

question is decidable

Esri/link-what-you-include (CMake)

- Flags declared deps that are never included
- Flags missing deps supplied only transitively

Layering Checks (Bazel)

- Same precision + public/private via `implementation_deps`

IWYU's core idea lifted one level up: apply checkability to the target, not just the file.

Defend it

Your build graph is your architecture

01. WIKI DIAGRAM

Aspirational

documentation drift

- Static, manually created, and quickly outdated.
- Represents intended design rather than execution reality.

02. THE BUILD DAG

Actual Reality

what actually ships

- Evaluated dynamically on every compilation and link.
- Defines true boundaries, inputs, outputs, and flow.

Where they disagree, the DAG is right.

Layering violations are graph facts

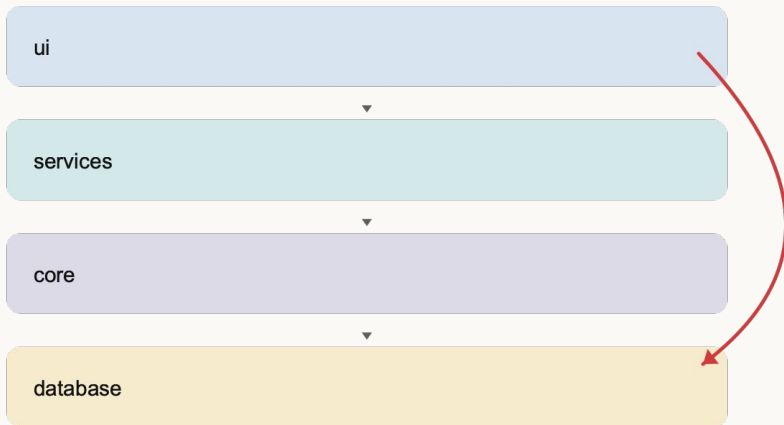
01. AUTOMATED CHECKS

Queryable Rules

"UI must not depend on database"

- Evaluated as a reachability query on the DAG
- Returns true or false in milliseconds
- No manual code review required

02. ILLEGAL DEPENDENCY



Ninja enforces acyclicity between files. Nobody enforces layering between targets — unless you do.

Four checks worth putting in CI

01. LATENCY GUARD

Critical Path Budget

fail condition

fail if the longest chain grows $> N \%$

02. BLAST RADIUS

Fan-in Cap

fail condition

fail if any header's rebuild set exceeds N targets

03. ARCHITECTURE RULE

Banned Edges

fail condition

fail if layer A becomes reachable from layer B

04. SCALABILITY FLOOR

Parallelism Floor

fail condition

fail if $\text{work} \div \text{critical path}$ drops below core count

Every one is computable from artifacts the build already produces.

Make the graph a reviewable artifact

01. AUTOMATED REPORTS

CI & PR Integration

continuous visibility

- Attach the report to every CI run
- Diff the critical path across a pull request
- Put the number in the PR description

02. ACTIONABLE FEEDBACK

Concrete Review Items

human-actionable insights

- "This change adds 4 s to the critical path" is a review comment
- gives reviewers something somebody can actually act on

Turn build performance metrics into clear, actionable pull request decisions.

The workflow, end to end

01. RECORD

Build Clean

`keep .ninja_log`

Start clean to collect complete baseline execution data.

02. METRICS

Read 3 Numbers

`baseline summary`

Evaluate critical build metrics to locate performance bottlenecks.

03. FOCUS

Critical Path

`parallelism < cores`

Focus on the longest target chain rather than individual slow files.

04. DIAGNOSE

Structural Cause

`root cause analysis`

Locate hub headers, coarse dependencies, or serialized targets.

05. REFACTOR

Change Graph

`DAG optimization`

Restructure build target graph dependencies to unlock parallel execution.

06. ENFORCE

Lock it in CI

`re-measure & guard`

Re-measure gains and lock down graph invariants against regressions.

Re-measure after every change. Slack moves.

Run this on your own build tonight

CLI COMMAND

```
python ninjascope.py path/to/build -o report.html
```

01. PREREQUISITES

Build Artifacts

clean build log

Needs build.ninja, .ninja_log, and a clean build.

02. COMPATIBILITY

Any Ninja Generator

CMake • GN • Meson

Works seamlessly with CMake, GN, Meson, or any ninja build directory.

03. ZERO DEPENDENCIES

Instant Setup

stdlib-only

Standard library only with no installation required; outputs a single self-contained HTML file.

Zero install, single-file HTML report — profile your Ninja build performance in seconds.

What you're leaving with

01. VISION

Build Graph Vision

your build is a DAG

Not a folder of files — nodes, edges, a critical path.
`build.ninja` & `.ninja_log` are already on disk.

02. METRICS

Hardware Metrics

$\text{work} \div \text{critical path}$

Three numbers decide if hardware helps: total work, critical path, and ratio. If below core count, next machine idles.

03. OPTIMIZATION

Graph Optimization

change shape, not speed

Shorten path, remove edges, collapse them — re-measure as slack moves. A single `PRIVATE` changes shape.

04. PREVENTION

Regression Prevention

measure & defend

Critical-path budget, fan-in cap, banned edges, parallelism floor — ratcheted to fail strictly on regressions.

The machine is not the problem. The graph is.

My DAG ate all my cores

It doesn't have to.

GITHUB REPOSITORY

`github.com/Orphis/NinjaScope`

Extra: reading list

01. DEPENDENCIES

[link-what-you-include](#)

Deps that match your includes

03. ECOSYSTEMS

[Bazel's Performance Metrics](#)

The same idea in another ecosystem

05. BUILD SYSTEM

[CMake Instrumentation](#) + Perfetto

Timeline view directly from CMake

02. THEORY

[Dilworth's theorem](#) & antichain width

The parallelism ceiling, formally

04. COMPILER TRACING

clang -ftime-trace & [ClangBuildAnalyzer](#)

Inside a single translation unit

06. NINJA TRACING

[ninjatracng](#) + Perfetto

Timeline view without NinjaScope

Recommended references & tools for analyzing build performance and critical paths.